PRACTICAL FILE

Analysis and Design of Algorithms

◆ ADA ◆

Algorithms | Complexity | Optimisation

Subject Code: 451

STUDENT NAME	ROLL NUMBER
Karanvir	UEM243135
BATCH	**SEMESTER**
2024 – 2028	4th Semester
SUBJECT CODE	**SUBJECT NAME**
451	Design & Analysis of Algorithms
UNIVERSITY	**SUBMITTED TO**
Panjab University, Chandigarh	Prof. Mukesh Kumar Rai

Academic Session: 2024 – 2025

B.E. / B.Tech | 4th Semester | Batch 2024

INDEX

Sr. No.	Week	Program / Topic	Time Complexity
01	Week 1	Velocity Calculation and Horner's Rule	O(n)
02		Linear Search	O(n)
03		Missing Number	O(n)
04		Permutation of Binary String	$O(2^n)$
05		Permutation of String	O(n!)
06		Power Iterative	O(n)
07		Power Recursive	O(log n)
08		Sum of Arrays	O(n)
09		Tower of Hanoi	$O(2^n)$
10	Week 2	Binary Search	O(log n)
11		Insertion Sort	$O(n^2)$
12	Week 3	Merge Sort	O(n log n)
13		Quick Sort	O(n log n) avg
14	Week 4	Convex Hull	$O(n^4)$
15		Fractional Knapsack	O(n log n)
16		Matrix Multiplication (Divide & Conquer)	$O(n^3)$
17	Week 5	Dijkstra's Algorithm	$O(V^2)$
18		Kruskal's Algorithm	O(E log E)
19		Prim's Algorithm	$O(V^2)$
20	Week 6	Cycle Detection in Graph (DFS)	O(V + E)
21	Week 7	Multistage Graph (Dynamic Programming,Counting stages,Vertices,Minpath)	O(V + E)
22	Week 8	Matrix Chain Multiplication	$O(n^3)$
23		Multistage Graph (Forward & Backward)	$O(V^2)$
24	Week 9	Floyd-Warshall (All Pairs Shortest Path)	$O(V^3)$
25		Longest Common Subsequence (LCS)	O(m x n)
26	Week 10	Travelling Salesman Problem (TSP)	$O(2^n \times n^2)$

Sr. No.	Week	Program / Topic	Time Complexity
27		Graph Colouring	$O(m^n)$
28		Hamiltonian Cycle	$O(n!)$
29		N-Queens Problem	$O(n!)$
30		Sum of Subsets	$O(2^n)$

DAA Lab Work - Design and Analysis of Algorithms

UEM243135

- **WEEK 1:**

Program 1: Ball Tips / Velocity Bounce Problem (`velocity.cpp`)

Aim

To calculate the number of times a ball will bounce until its velocity drops below a minimum threshold using recursion.

Pseudocode

```
ALGO CountBounces(v, t)

   IF v >= 1

      t ← t + 1

      v ← v - (0.425 × v)

      RETURN CountBounces(v, t)

   ELSE

      RETURN t

   END IF
```

Variables Used

- `v` — Current velocity of the ball

- `t` — Counter storing number of bounces

- `0.425` — Resistance/reduction factor (42.5% velocity lost per bounce)

Algorithm Explanation

The algorithm recursively reduces velocity after each bounce.

When velocity drops below 1 (threshold), recursion stops and total bounces are returned.

Each recursive call increments the bounce counter by 1.

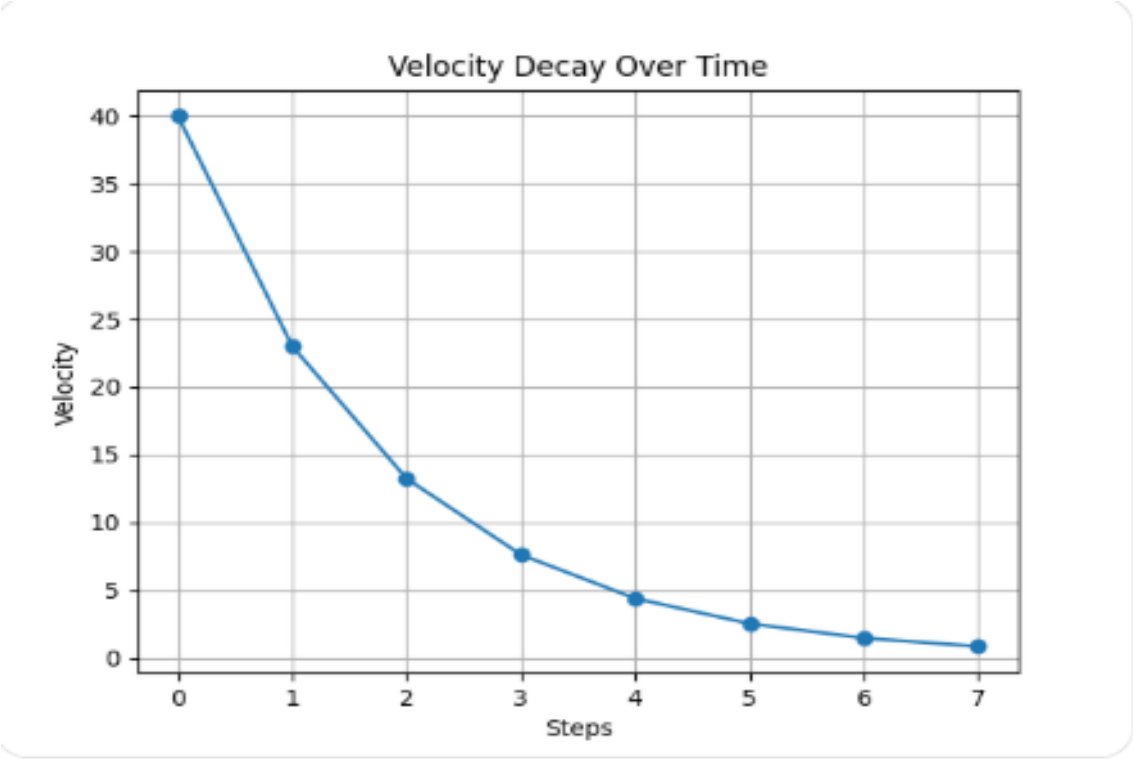
Use Cases

- Used in **sports science simulations** to model how many times a ball bounces before stopping.

- Helpful in **physics engines** for game development where object rebound behaviour needs to be calculated.

- Can be applied in **robotics** to predict how many hops a robot or drone makes before landing.

Graph



Time Complexity

O(log n)

Space Complexity

O(log n)

Program 2: Horner's Rule (`HornersRule.cpp`)

Aim

To evaluate a polynomial efficiently using Horner's method, reducing the number of multiplications, and measure average execution time for varying polynomial degrees.

Pseudocode

```
ALGO HornersRule(coef[], n, x)

   result ← coef[n]

   FOR i = n-1 DOWN TO 0

      result ← coef[i] + x × result

   END FOR

   RETURN result
```

Variables Used

- `coef[]` — Array of polynomial coefficients

- `n` — Degree of the polynomial

- `x` — Value at which polynomial is evaluated

- `result` — Accumulated result of evaluation

Algorithm Explanation

Horner's Rule evaluates a degree-n polynomial using only n multiplications and n additions.

Instead of computing each term separately, it nests the evaluation from highest degree down.

This avoids redundant power computations.

Use Cases

- Used in **compilers and calculators** to efficiently evaluate polynomials at runtime.

- Applied in **cryptography** for fast polynomial hashing (e.g. Rabin-Karp string search).

- Useful in **numerical analysis** software where polynomial evaluation is a frequent and performance-critical operation.

Graph

Time Complexity

O(n)

Space Complexity

O(1)

Program 3: Linear Search (`LinearSearch.cpp`)

Aim

To search for a target element in an unsorted array by scanning each element sequentially and measure average execution time for different array sizes.

Pseudocode

```
ALGO LinearSearch(arr[], n, target)

   FOR i = 0 TO n-1

      IF arr[i] = target

         RETURN i

      END IF

   END FOR

   RETURN -1
```

Variables Used

- `arr[]` — Input array

- `n` — Size of the array

- `target` — Element to search for

- `i` — Loop index

Algorithm Explanation

Linear search scans each element of the array one by one.

If the target is found, its index is returned.

If the entire array is scanned without finding the target, -1 is returned.

Worst case occurs when target is absent (scans all n elements).

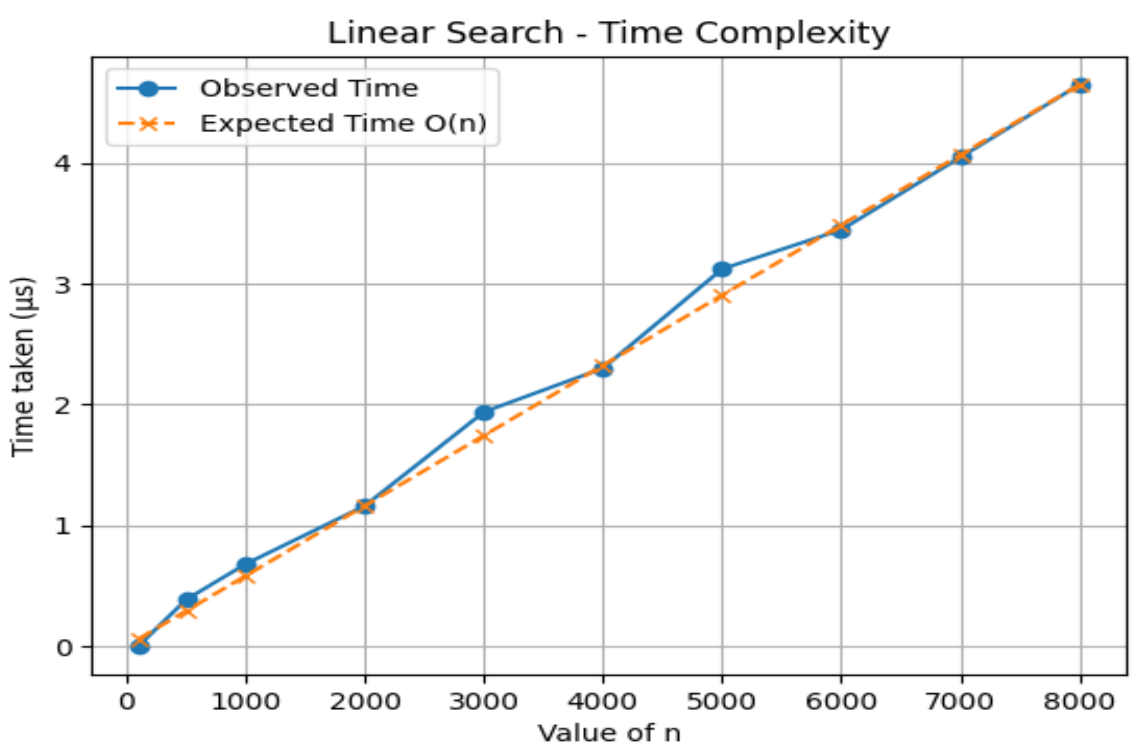
Use Cases

- Used to search **unsorted or small datasets** where sorting overhead is not justified.

- Applied in **embedded systems** with limited memory where binary search setup is not feasible.

- Commonly used in **debugging or validation** to verify correctness before applying optimised search methods.

Graph



Time Complexity

O(n)

Space Complexity

O(1)

Program 4: Missing Number (`MissingNumber.cpp`)

Aim

To find the missing number in a sorted consecutive integer array by detecting where the difference between adjacent elements is not 1.

Pseudocode

```
ALGO FindMissingNumber(arr[], n)

   FOR i = 1 TO n-1

      IF arr[i] - arr[i-1] ≠ 1

         RETURN arr[i-1] + 1

      END IF

   END FOR

   RETURN arr[n-1] + 1
```

Variables Used

- `arr[]` — Sorted array with one missing number

- `n` — Size of the array

- `i` — Loop index

Algorithm Explanation

The algorithm checks consecutive differences between elements.

If the difference between two adjacent elements is more than 1, the missing number lies between them.

If no gap is found inside the array, the missing number is the next number after the last element.

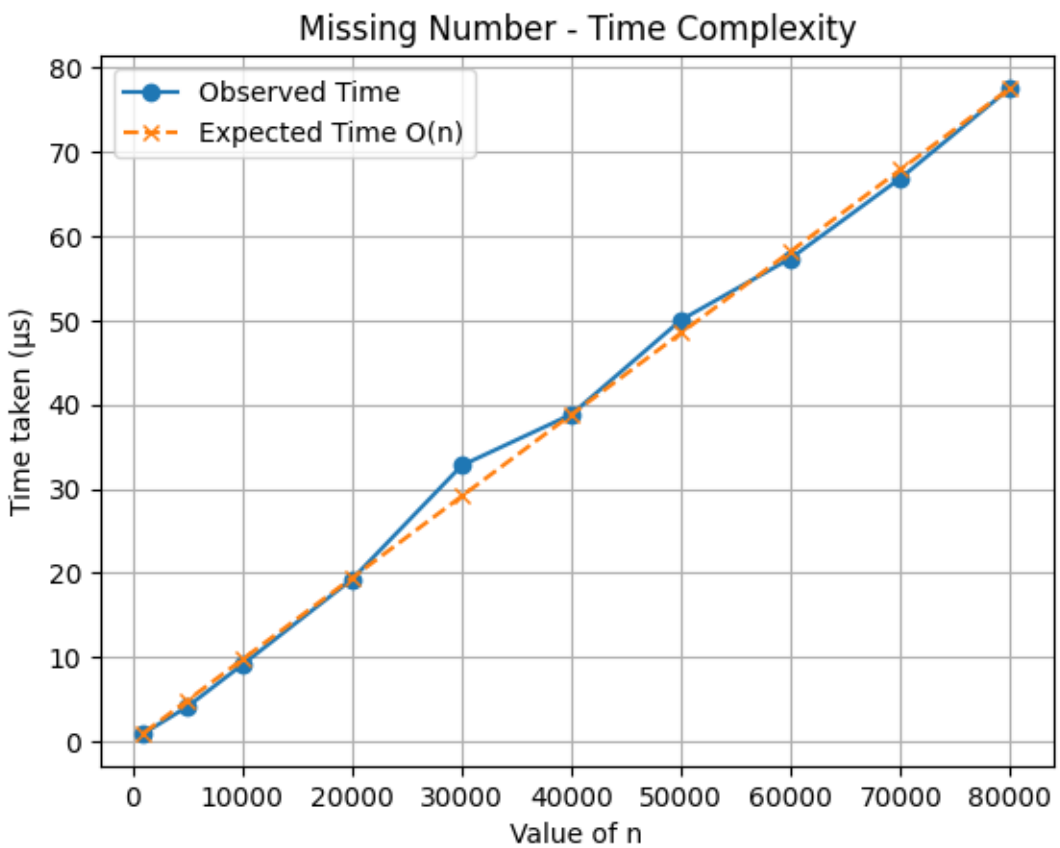
Use Cases

- Used in **data integrity checks** to detect missing records in sequential ID systems (e.g. invoice or ticket numbers).

- Applied in **attendance or roll-call systems** to find which serial entry was skipped.

- Useful in **exam/result processing** where roll numbers must be consecutive and any gap signals a missing student.

Graph



Time Complexity

O(n)

Space Complexity

O(1)

Program 5: Permutation of Binary String (`PermutationBinaryString.cpp`)

Aim

To generate all possible binary strings of length n using recursion, assigning '0' and '1' at each position.

Pseudocode

```
ALGO Permutation(current[], n, idx)
    IF idx = n
        RETURN
    END IF
    current[idx] ← '0'
    Permutation(current, n, idx+1)
    current[idx] ← '1'
    Permutation(current, n, idx+1)
```

Variables Used

- `current[]` — Character array holding the current binary string
- `n` — Length of the binary string
- `idx` — Current position being filled

Algorithm Explanation

At each position, the algorithm places '0' and recurses, then places '1' and recurses.

This generates all 2^n binary strings of length n.

The base case is when idx equals n meaning all positions are filled.

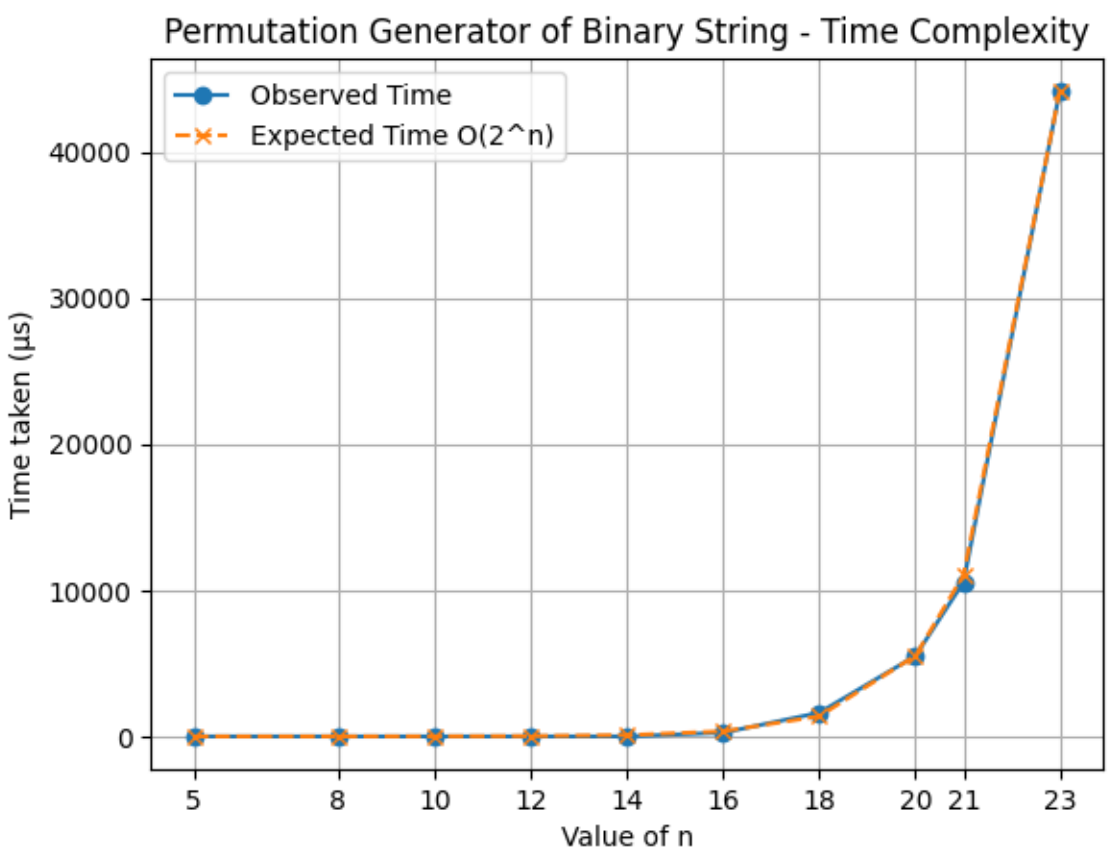
Use Cases

- Used in **cryptography and security** to generate all possible bit patterns for brute-force key testing.

- Applied in **digital circuit design** to enumerate all possible input combinations for logic gate testing.

- Helpful in **competitive programming** for generating all binary subsets or states in bitmask problems.

Graph



Time Complexity

$O(2^n)$

Space Complexity

O(n)

Program 6: Permutation of String (`PemutationString.cpp`)

Aim

To generate all permutations of a given character array using backtracking by swapping characters at each position.

Pseudocode

```
ALGO Permutation(arr[], start, n)

   IF start = n

      RETURN

   END IF

   FOR i = start TO n-1

      SWAP arr[start] ↔ arr[i]

      Permutation(arr, start+1, n)

      SWAP arr[start] ↔ arr[i]   // backtrack

   END FOR
```

Variables Used

- `arr[]` — Character array to permute

- `start` — Current starting position

- `n` — Length of the string

- `i` — Loop index for swapping

Algorithm Explanation

At each recursive call, the current position is fixed with each remaining character via swapping.

After recursion, the swap is undone (backtracking) to restore the original order.

This generates all n! permutations of the string.

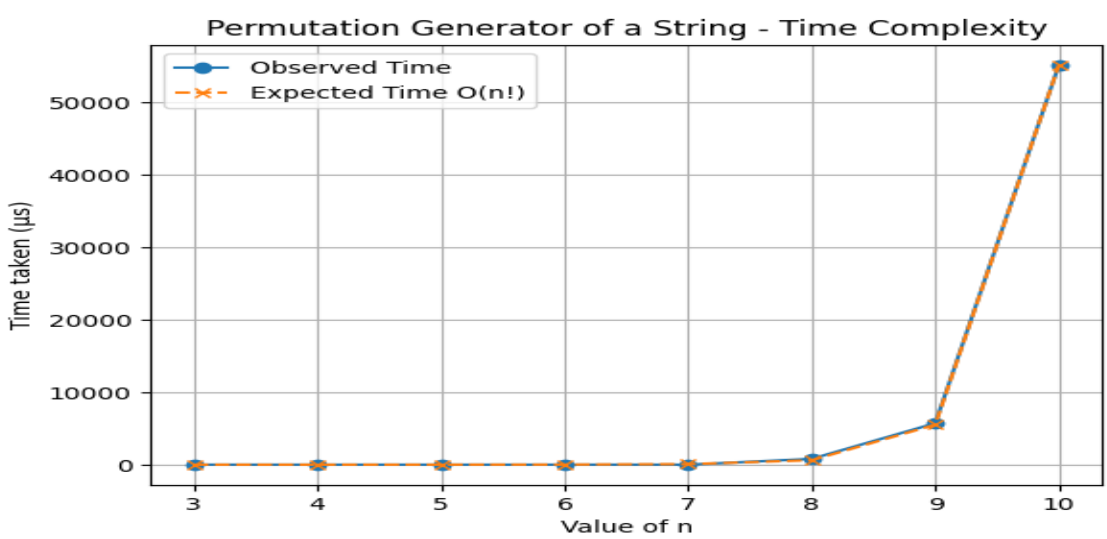
Use Cases

- Used in ****password recovery tools**** to generate all possible character arrangements for brute-force attempts.

- Applied in ****game development**** for anagram-based puzzles where all word arrangements must be explored.

- Useful in ****combinatorics research**** and test case generation where exhaustive string enumeration is needed.

###Graph



Time Complexity

O(n!)

Space Complexity

O(n)

Program 7: Power Iterative (`PowerIterative.cpp`)

Aim

To compute x raised to the power n using an iterative loop and measure average execution time for different exponent values.

Pseudocode

```
ALGO PowerIterative(x, n)

   result ← 1

   FOR i = 0 TO n-1

      result ← result × x

   END FOR

   RETURN result
```

Variables Used

- `x` — Base value

- `n` — Exponent value

- `result` — Accumulated product

- `i` — Loop index

Algorithm Explanation

The algorithm multiplies x by itself n times iteratively.

Simple and straightforward but less efficient than recursive approach for large n.

Result accumulates through n multiplications.

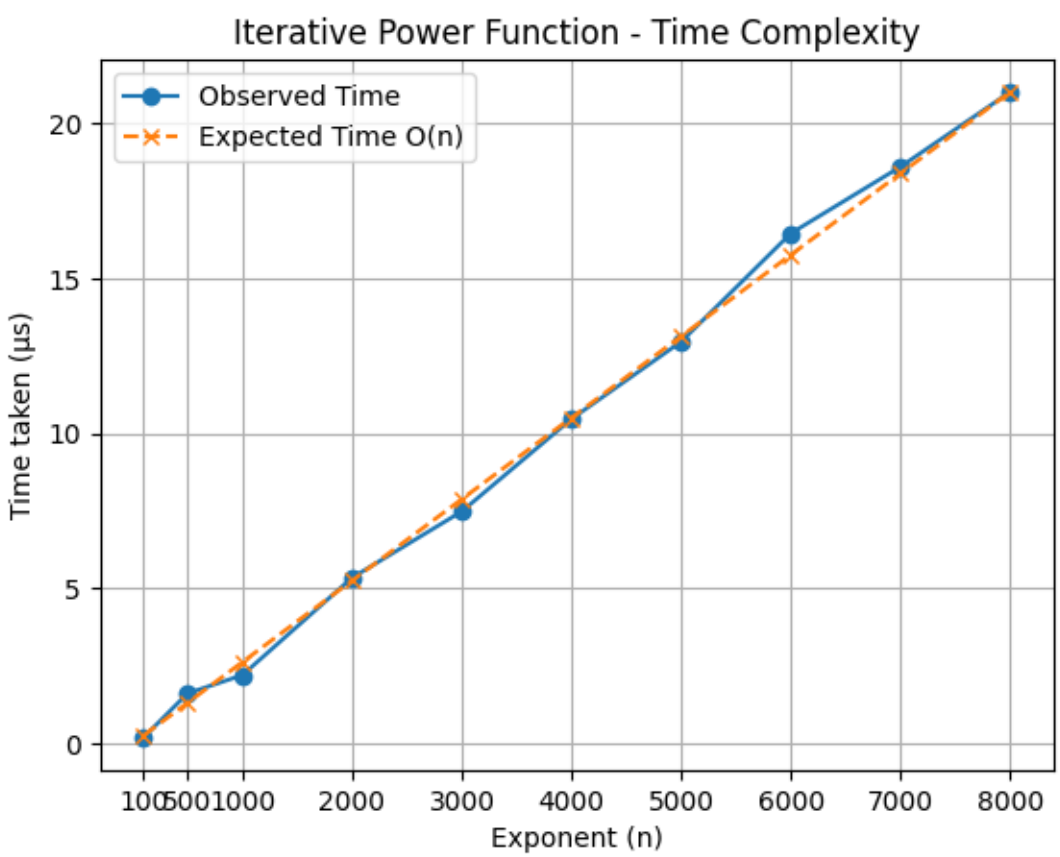
Use Cases

- Used in **basic scientific calculators** and educational tools to compute integer powers step by step.

- Applied as a **baseline benchmark** to compare against faster recursive or library-based exponentiation methods.

- Useful in **teaching environments** to demonstrate loop-based problem solving before introducing divide and conquer.

###Graph



Time Complexity

O(n)

Space Complexity

O(1)

Program 8: Power Recursive (`PowerRecursive.cpp`)

Aim

To compute x raised to the power n using divide and conquer (fast exponentiation) recursively and measure average execution time.

Pseudocode

```
ALGO PowerRecursive(x, n)

   IF n = 0

      RETURN 1

   END IF

   half ← PowerRecursive(x, n/2)

   IF n is EVEN

      RETURN half × half

   ELSE

      RETURN x × half × half

   END IF
```

Variables Used

- `x` — Base value

- `n` — Exponent value

- `half` — Result of recursive half-power call

Algorithm Explanation

The algorithm uses divide and conquer splitting the exponent in half each time.

If n is even: $x^n = (x^{(n/2)})^2$

If n is odd: $x^n = x \times (x^{(n/2)})^2$

This reduces multiplications from n to log n.

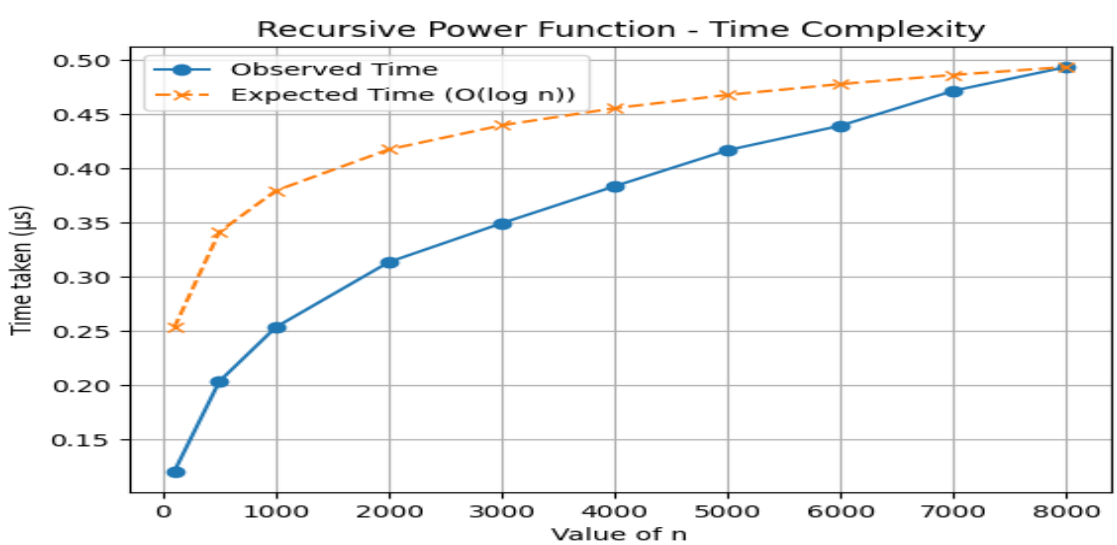
Use Cases

- Used in **cryptographic algorithms** like RSA where modular exponentiation with large numbers is required.

- Applied in **competitive programming** to compute large powers efficiently within time constraints.

- Useful in **numerical computing libraries** to speed up repeated power calculations in simulations.

Graph



Time Complexity

O(log n)

Space Complexity

O(log n)

Program 9: Sum of Arrays (`Sumofarrays.cpp`)

Aim

To compute the sum of all elements in an array using a simple loop and measure average execution time for different array sizes.

Pseudocode

```
ALGO SumOfArray(arr[], n)

   sum ← 0

   FOR i = 0 TO n-1

      sum ← sum + arr[i]

   END FOR

   RETURN sum
```

Variables Used

- `arr[]` — Input array of integers

- `n` — Size of the array

- `sum` — Accumulated total

- `i` — Loop index

Algorithm Explanation

The algorithm sequentially visits every element and adds it to the running sum.

After all elements are processed, the total sum is returned.

Simple O(n) linear scan with no extra space needed.

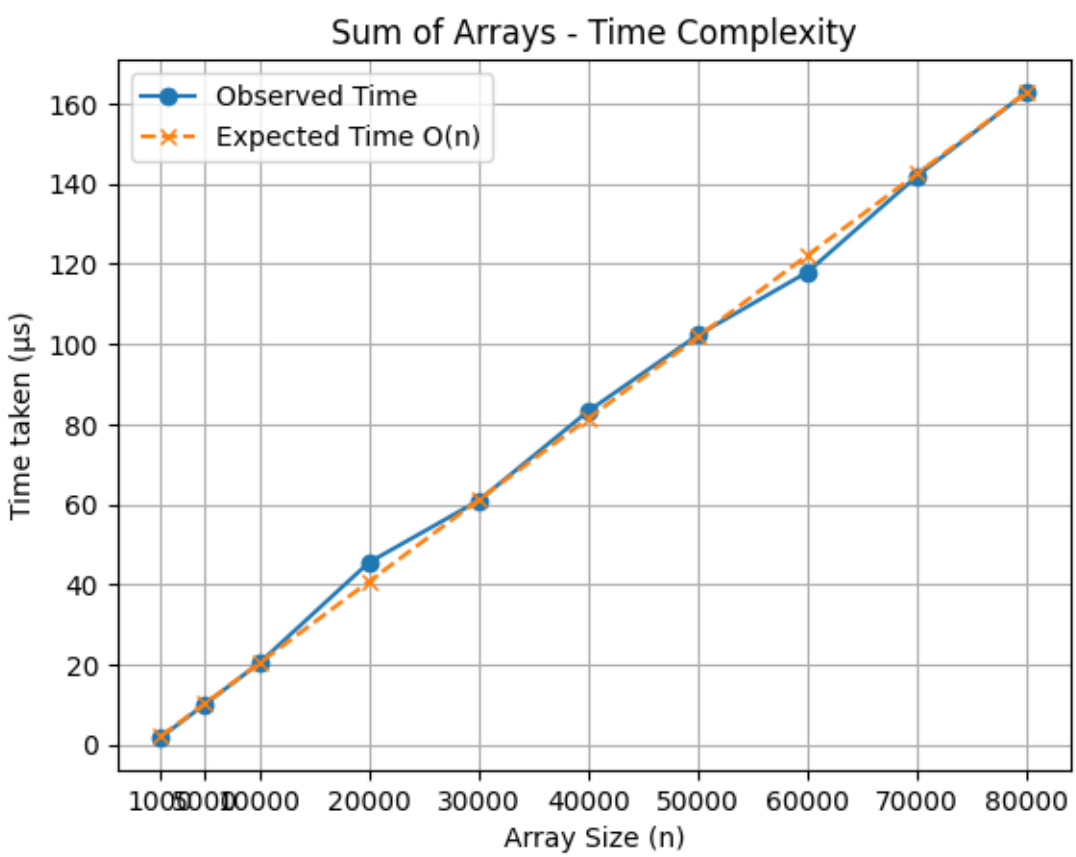
Use Cases

- Used in **data analytics pipelines** for aggregating values across large datasets (e.g. total sales, scores).

- Applied in **sensor data systems** to compute cumulative readings from IoT devices over time.

- Fundamental operation used in **matrix operations, statistics, and signal processing** as a building block.

###Graph



Time Complexity

O(n)

Space Complexity

O(1)

Program 10: Tower of Hanoi (`TowersofHanoi.cpp`)

Aim

To solve the Tower of Hanoi problem for n disks using recursion and measure average execution time for different disk counts.

Pseudocode

```
ALGO TowerOfHanoi(n, from, to, aux)

    IF n = 0

        RETURN

    END IF

    TowerOfHanoi(n-1, from, aux, to)

    MOVE disk n FROM 'from' TO 'to'

    TowerOfHanoi(n-1, aux, to, from)
```

Variables Used

- `n` — Number of disks

- `from` — Source peg

- `to` — Destination peg

- `aux` — Auxiliary/helper peg

Algorithm Explanation

Move n-1 disks from source to auxiliary peg using destination as helper.

Move the largest (nth) disk directly from source to destination.

Move n-1 disks from auxiliary to destination peg using source as helper.

Total moves required = 2^n - 1.

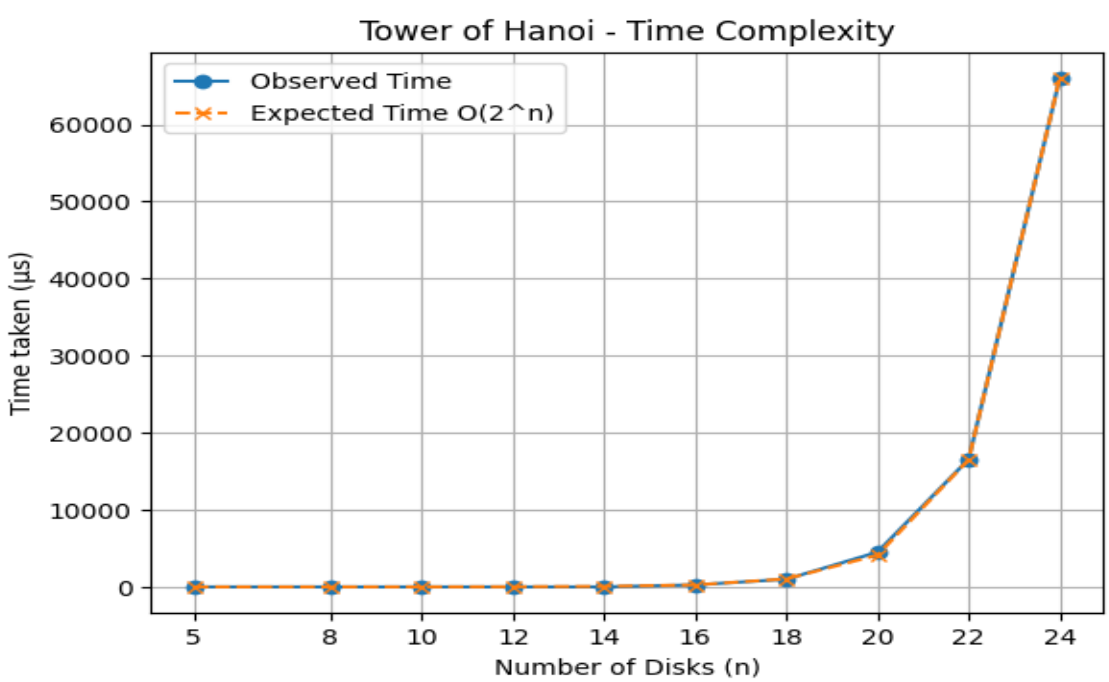
Use Cases

- Used in **computer science education** to teach recursion, stack behaviour, and exponential time complexity.

- Applied in **backup and migration systems** conceptually — moving data in layers without conflicts.

- Studied in **algorithm analysis** as a classic example of a problem with provably optimal recursive structure.

###Graph



Time Complexity

$O(2^n)$

Space Complexity

O(n)

- **WEEK 2:**

Program 1: Binary Search (`BinarySearch.cpp`)

Aim

To search for a target element in a sorted array using the divide and conquer approach and measure average execution time for different array sizes.

Pseudocode

```
ALGO BinarySearch(arr[], low, high, target)

   IF low > high

      RETURN -1

   END IF

   mid ← (low + high) / 2

   IF arr[mid] = target

      RETURN mid

   ELSE IF arr[mid] > target

      RETURN BinarySearch(arr, low, mid-1, target)
```

```
    ELSE
        RETURN BinarySearch(arr, mid+1, high, target)
    END IF
```

Variables Used

- `arr[]` — Sorted input array
- `low` — Left boundary of search range
- `high` — Right boundary of search range
- `mid` — Middle index of current range
- `target` — Element to search for

Algorithm Explanation

Binary search repeatedly halves the search space.

At each step, the middle element is compared with the target.

If equal, index is returned. If target is smaller, search left half. If larger, search right half.

Continues until element is found or range becomes empty.

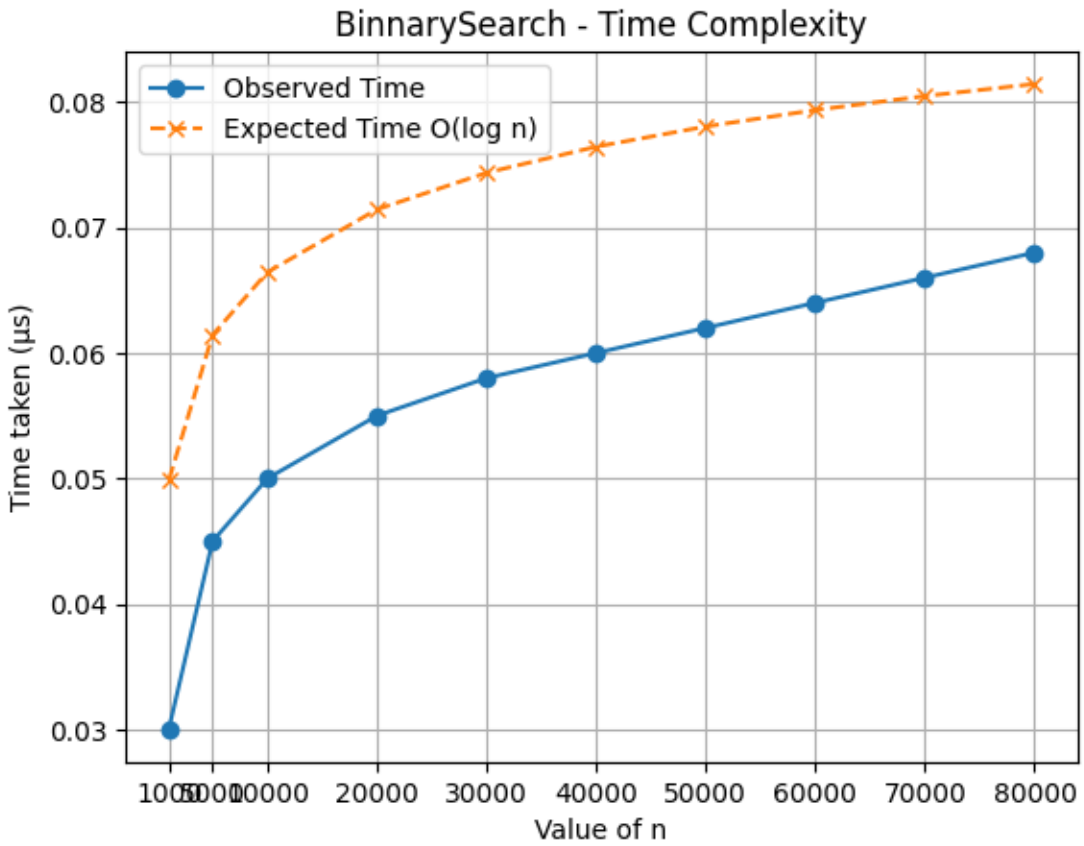
Use Cases

- Used in **database indexing systems** to quickly locate records in sorted tables without scanning all rows.

- Applied in **dictionary or autocomplete features** to find words efficiently in alphabetically sorted lists.

- Widely used in **competitive programming and system utilities** such as `git bisect` to find the commit introducing a bug.

###Graph



Time Complexity

O(log n)

Space Complexity

O(log n)

<u>## Program 2: Insertion Sort (`InsertionSort.cpp`)</u>

Aim

To sort an array in ascending order using the Insertion Sort algorithm and measure worst-case average execution time for different array sizes.

Pseudocode

```
ALGO InsertionSort(arr[], n)

   FOR i = 1 TO n-1

      key ← arr[i]

      j ← i - 1

      WHILE j >= 0 AND arr[j] > key

         arr[j+1] ← arr[j]

         j ← j - 1

      END WHILE

      arr[j+1] ← key

   END FOR
```

Variables Used

- `arr[]` — Input array

- `n` — Size of the array

- `key` — Current element being inserted

- `j` — Index for backward comparison

Algorithm Explanation

Insertion sort builds the sorted array one element at a time.

For each element, it finds the correct position by shifting larger elements rightward.

The element is then inserted at its correct sorted position.

Works like sorting a hand of playing cards — pick one card and insert in right place.

Use Cases

- Used in **real-time systems** where data arrives incrementally and needs to stay sorted at all times.

- Applied for **small datasets or nearly sorted arrays** where overhead of merge/quick sort is unnecessary.

- Used in **hybrid sorting algorithms** like TimSort (Python, Java) as the base case for small partitions.

Graph

Time Complexity

$O(n^2)$

Space Complexity

O(1)

- **WEEK 3:**

Program 1: Merge Sort (`mergesort.cpp`)

Aim

To sort an array using the Merge Sort algorithm (divide and conquer) and measure average execution time for different array sizes.

Pseudocode

```
ALGO MergeSort(A[], p, r)

   IF p < r

      q ← (p + r) / 2

      MergeSort(A, p, q)

      MergeSort(A, q+1, r)

      Merge(A, p, q, r)

   END IF

ALGO Merge(A[], p, q, r)

   Create L[0..n1] and R[0..n2]

   Copy A[p..q] into L[]

   Copy A[q+1..r] into R[]
```

```
L[n1] ← ∞,  R[n2] ← ∞     // sentinels
i ← 0, j ← 0
FOR k = p TO r
    IF L[i] ≤ R[j]
        A[k] ← L[i],  i ← i+1
    ELSE
        A[k] ← R[j],  j ← j+1
    END IF
END FOR
```

Variables Used

- `A[]` — Input array
- `p` — Left index of subarray
- `r` — Right index of subarray
- `q` — Middle index (split point)
- `L[], R[]` — Temporary left and right subarrays
- `i, j, k` — Indices for merging

Algorithm Explanation

Merge Sort divides the array into two halves recursively until each half has one element.

Then it merges sorted halves back together in correct order.

Sentinel value (INT_MAX) is placed at end of L and R to avoid bounds checking.

Always splits evenly so depth of recursion tree is log n.

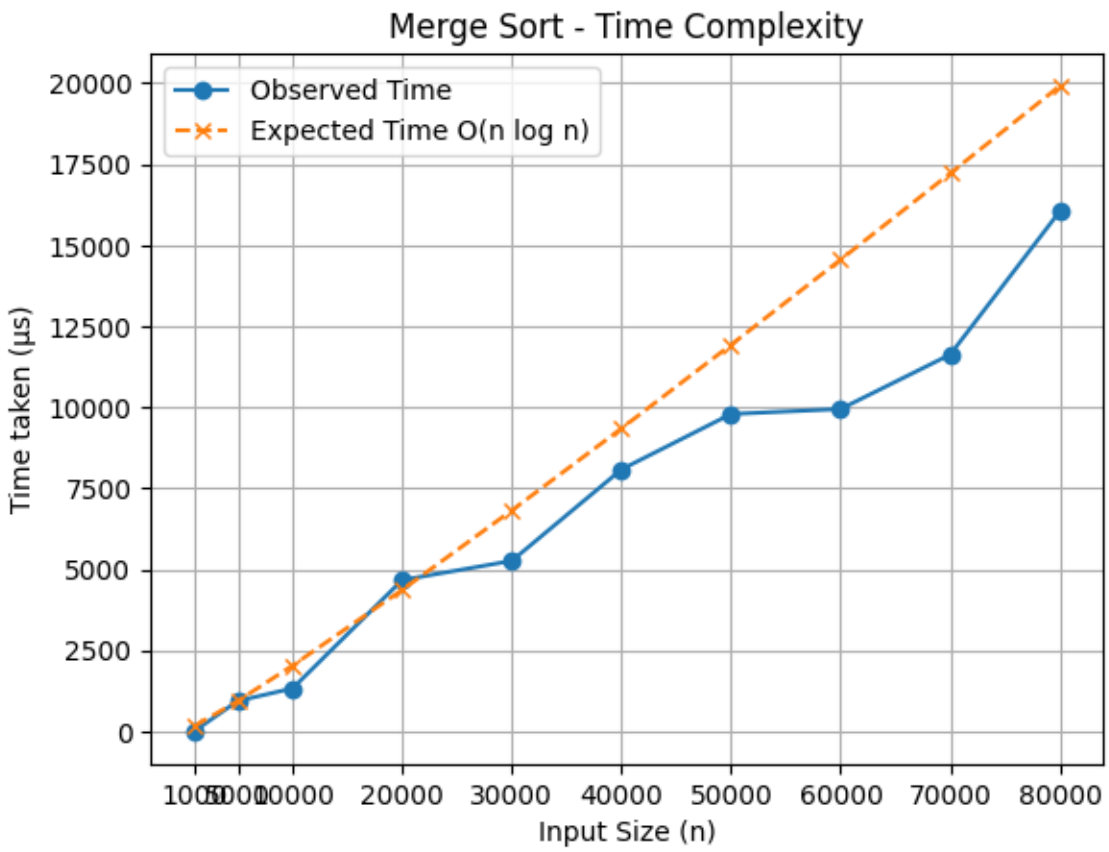
Use Cases

- Used in **external sorting** where data is too large to fit in RAM and must be sorted in chunks from disk.

- Applied in **linked list sorting** where merge sort is preferred due to no random access requirement.

- Used in **parallel computing** frameworks where the merge step can be distributed across multiple processors.

###Graph



Time Complexity

O(n log n)

Space Complexity

O(n

Program 2: Quick Sort (`quicksort.cpp`)

Aim

To sort an array using the Quick Sort algorithm with first element as pivot and measure average execution time for different array sizes.

Pseudocode

```
ALGO QuickSort(A[], p, q)

   IF p < q

      pivot ← Partition(A, p, q)

      QuickSort(A, p, pivot-1)

      QuickSort(A, pivot+1, q)

   END IF

ALGO Partition(A[], p, q)

   x ← A[p]       // pivot = first element

   i ← p

   FOR j = p+1 TO q

      IF A[j] ≤ x

         i ← i + 1

         SWAP A[i] ↔ A[j]

      END IF
```

END FOR

SWAP A[i] ↔ A[p]

RETURN i

Variables Used

- `A[]` — Input array

- `p` — Left boundary

- `q` — Right boundary

- `x` — Pivot element (first element)

- `i` — Partition boundary index

- `j` — Scanning index

Algorithm Explanation

Quick Sort picks first element as pivot and partitions array around it.

All elements smaller than pivot go left, larger go right.

Pivot is then placed in its final correct sorted position.

Recursively sorts left and right subarrays independently.

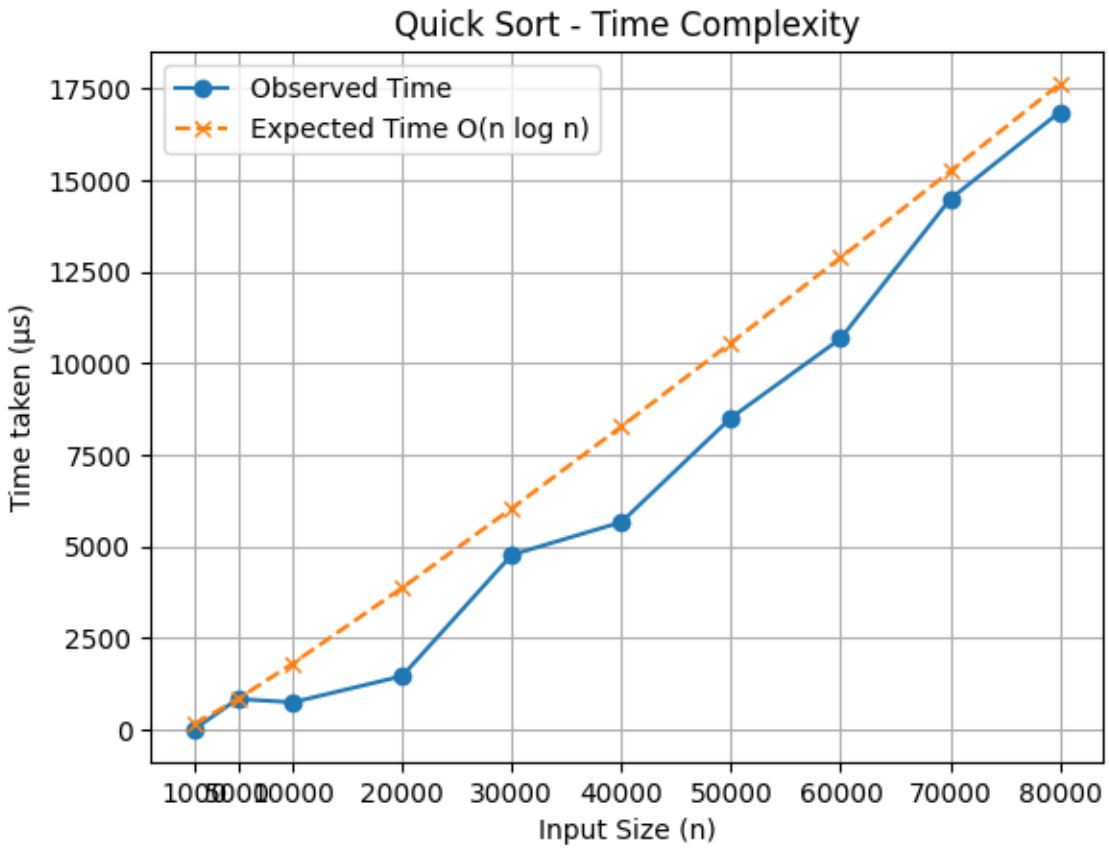
Use Cases

- Used in **standard library sorting** functions (e.g. C `qsort`, Java Arrays.sort for primitives) due to cache efficiency.

- Applied in **database query engines** for in-memory sorting of query result sets.

- Widely used in **competitive programming** for fast in-place sorting of large arrays within tight time limits.

###Graph



Time Complexity

O(n log n) average, O(n^2) worst case

Space Complexity

O(log n)

- **WEEK 4:**

Program 1: Convex Hull (`convexhull.cpp`)

Aim

To find the convex hull of a set of 2D points by identifying which points lie inside triangles formed by other points, and counting boundary points.

Pseudocode

```
ALGO ConvexHull(points[], n)

   isInterior[0..n-1] ← false

   FOR i = 0 TO n-1

      FOR j = i+1 TO n-1

         FOR k = j+1 TO n-1

            FOR p = 0 TO n-1

               IF p ≠ i,j,k AND PointInsideTriangle(i,j,k,p)

                  isInterior[p] ← true

               END IF

            END FOR

         END FOR

      END FOR

   END FOR

   count ← 0
```

```
    FOR i = 0 TO n-1
        IF isInterior[i] = false
            count ← count + 1
        END IF
    END FOR
    RETURN count


ALGO CrossProduct(A, B, C)
    RETURN (B.x-A.x)×(C.y-A.y) - (B.y-A.y)×(C.x-A.x)
```

Variables Used

- `points[]` — Array of 2D points with x and y coordinates

- `isInterior[]` — Boolean array marking interior points

- `boundaryCount` — Number of hull boundary points

Algorithm Explanation

For every combination of 3 points forming a triangle, check if any other point lies inside it.

Uses cross product to determine if a point lies inside a triangle.

Points that are interior to any triangle are excluded from the hull.

Remaining non-interior points form the convex hull boundary.

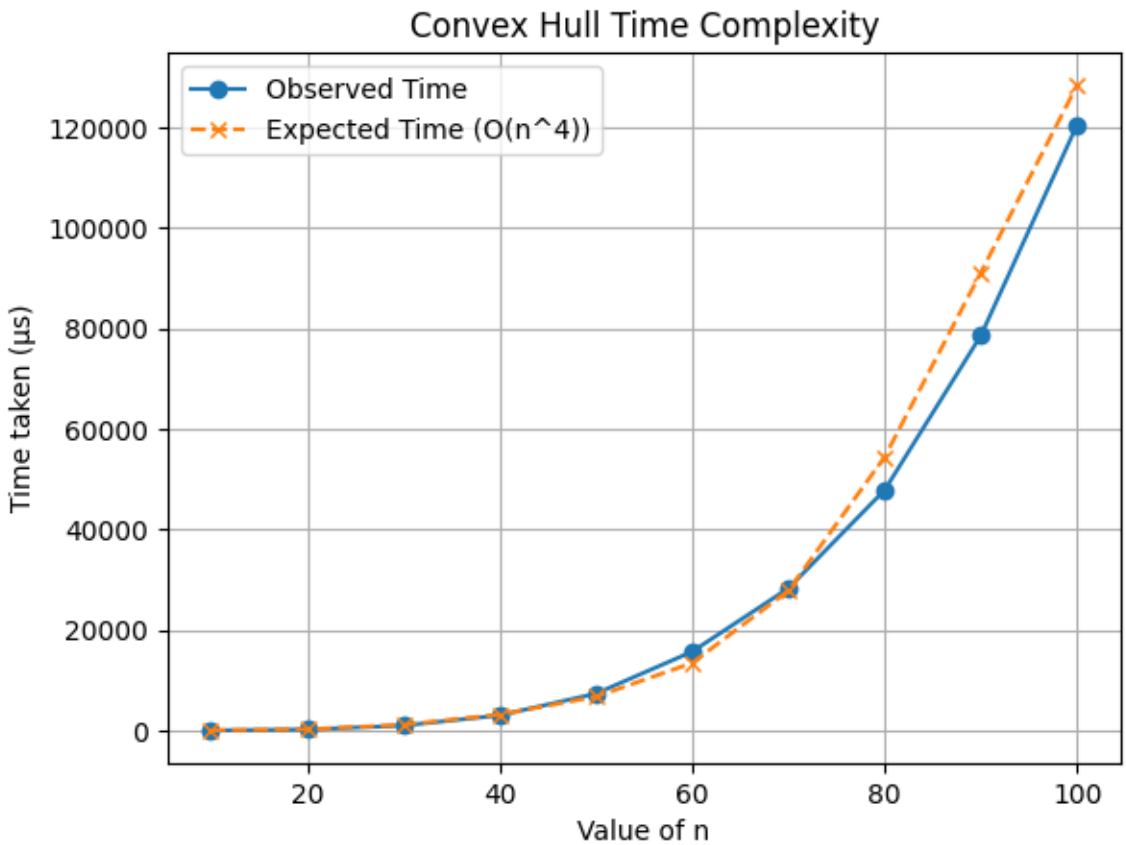
Use Cases

- Used in **computer graphics and game development** to compute bounding regions for collision detection.

- Applied in **robotics path planning** to define the navigable boundary of an obstacle-free region.

- Used in **GIS and mapping software** to compute the outer boundary of a cluster of GPS coordinates.

###Graph



Time Complexity

$O(n^4)$

Space Complexity

O(n)

<u>## Program 2: Fractional Knapsack (`FRACTIONALKNAPSACK.cpp`)</u>

Aim

To solve the Fractional Knapsack problem using three greedy strategies and compare their execution times.

Pseudocode

```
ALGO FractionalKnapsack(items[], n, M)

    SortByRatio(items, n)      // descending profit/weight ratio

    totalProfit ← 0.0

    FOR i = 0 TO n-1

        IF M ≥ items[i].weight

            totalProfit ← totalProfit + items[i].profit

            M ← M - items[i].weight

        ELSE

            fraction ← M / items[i].weight

            totalProfit ← totalProfit + fraction × items[i].profit

            BREAK

        END IF

    END FOR

    RETURN totalProfit
```

Variables Used

- `items[]` — Array of items with profit and weight fields

- `n` — Number of items

- `M` — Knapsack capacity

- `totalProfit` — Accumulated profit

- `fraction` — Fractional portion of last item

Algorithm Explanation

Items are sorted by profit-to-weight ratio in descending order.

Items are greedily picked — fully if capacity allows, fractionally for the last item.

Three strategies compared: sort by weight, sort by profit, sort by ratio.

Sorting by ratio gives the globally optimal result for fractional knapsack.

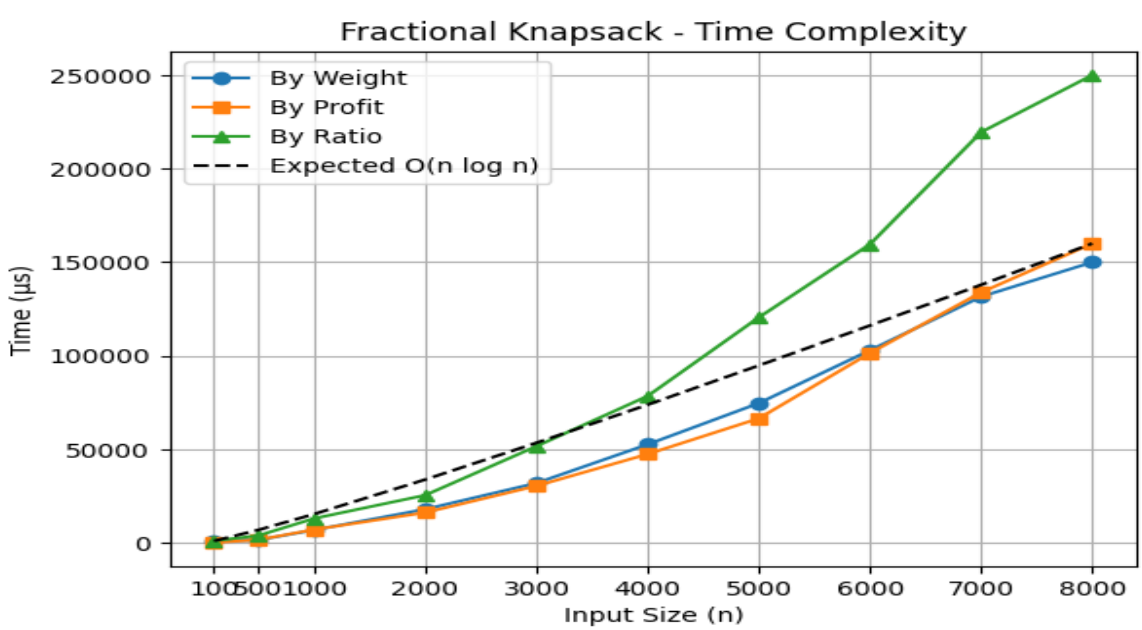
Use Cases

- Used in **resource allocation problems** like distributing CPU time or bandwidth across competing tasks.

- Applied in **financial portfolio management** to allocate capital across investments for maximum return per unit risk.

- Used in **cargo loading systems** to maximise profit when goods can be split (e.g. grain, liquid, sand).

###Graph



Time Complexity

O(n log n)

Space Complexity

O(1)

Program 3: Matrix Multiplication - Divide and Conquer (`DivConMatrix_Mul.cpp`)

Aim

To multiply two square matrices using divide and conquer by splitting each matrix into 4 submatrices and recursively multiplying them.

Pseudocode

```
ALGO MultiplyTables(mat1, mat2, res, sz)

   IF sz = 1

      res[0][0] ← mat1[0][0] × mat2[0][0]

      RETURN

   END IF

   half ← sz / 2

   Split mat1 → p11, p12, p21, p22

   Split mat2 → q11, q12, q21, q22

   r11 ← Multiply(p11,q11) + Multiply(p12,q21)

   r12 ← Multiply(p11,q12) + Multiply(p12,q22)

   r21 ← Multiply(p21,q11) + Multiply(p22,q21)
```

r22 ← Multiply(p21,q12) + Multiply(p22,q22)

Merge r11,r12,r21,r22 → res

Variables Used

- `mat1, mat2` — Input matrices

- `res` — Result matrix

- `sz` — Current matrix dimension

- `half` — Half of current size

- `p11..p22, q11..q22` — Quadrant submatrices

Algorithm Explanation

Each matrix is divided into four equal quadrants recursively.

8 recursive multiplications and 4 matrix additions are performed at each level.

Base case is 1×1 matrix — direct scalar multiplication.

Submatrices are split, multiplied recursively, and merged back.

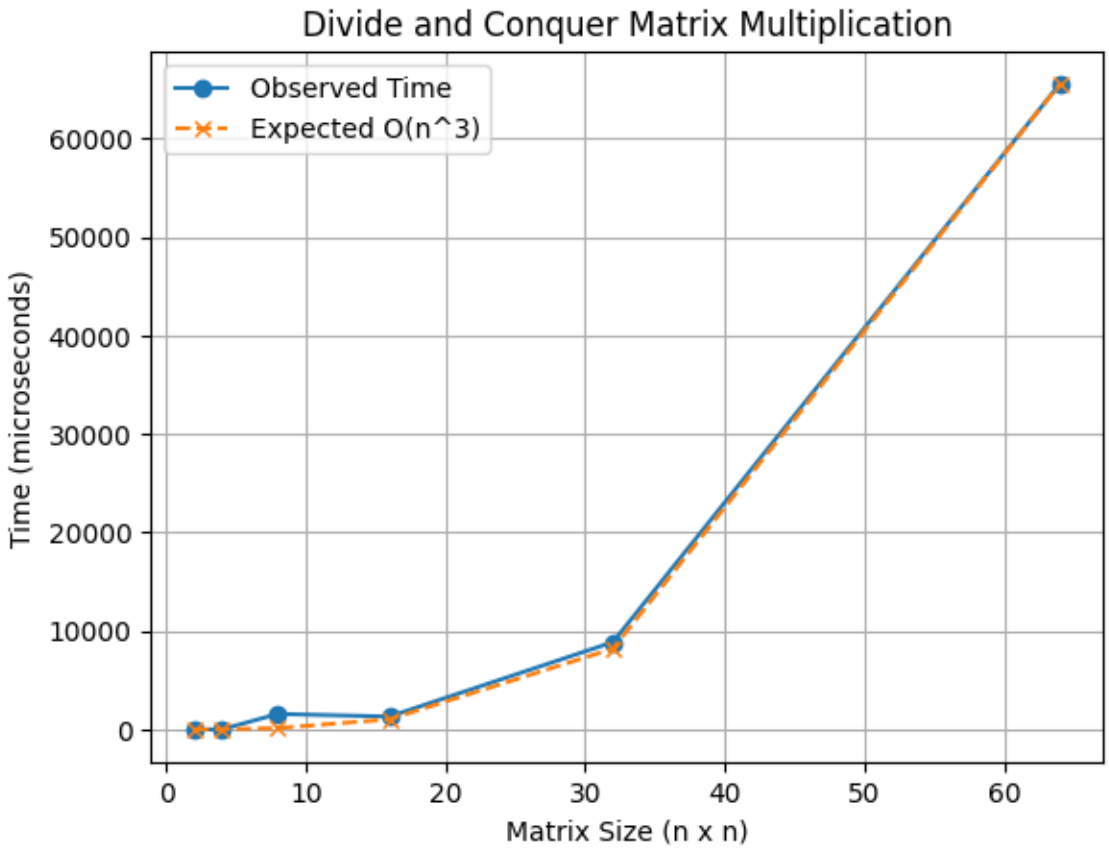
Use Cases

- Used in **machine learning** for computing weight updates and activations in neural networks.

- Applied in **computer graphics** for 3D transformation matrices (rotation, scaling, projection).

- Used in **scientific simulations** like finite element analysis and solving systems of linear equations.

###Graph



Time Complexity

$O(n^3)$

Space Complexity

$O(n^2)$

- **<u>WEEK 5:</u>**

<u>## Program 1: Dijkstra's Algorithm (`Dijkstra.cpp`)</u>

Aim

To find the shortest path from a single source vertex to all other vertices in a weighted graph using Dijkstra's algorithm with a custom min-heap.

Pseudocode

```
ALGO Dijkstra(n, source)

    dist[source] ← 0

    dist[all others] ← ∞

    Push (0, source) into MinHeap

    WHILE heap not empty

        (d, u) ← Extract minimum

        IF d ≠ dist[u] THEN CONTINUE

        FOR each neighbor v of u with weight w

            IF dist[u] + w < dist[v]

                dist[v] ← dist[u] + w

                Push (dist[v], v) into heap

            END IF

        END FOR

    END WHILE
```

Variables Used

- `dist[]` — Shortest distance from source to each vertex

- `source` — Starting vertex

- `d, u` — Distance and vertex extracted from heap

- `v, w` — Neighbor vertex and edge weight

Algorithm Explanation

Initialize all distances to infinity, source to 0.

Extract minimum distance vertex from heap and relax all its outgoing edges.

If a shorter path to neighbor v is found via u, update dist[v] and push to heap.

Stale entries in heap are skipped using the distance check (d != dist[u]).

Use Cases

- Used in **GPS navigation systems** like Google Maps to find the shortest driving route between two locations.

- Applied in **network routing protocols** (e.g. OSPF) to find lowest-cost paths between routers.

- Used in **game AI pathfinding** to navigate characters through weighted maps or terrain grids.

Time Complexity

O(V + E log V)

Space Complexity

O(V + E)

Program 2: Kruskal's Algorithm (`Krushkalalgo.cpp`)

Aim

To find the Minimum Spanning Tree (MST) of a weighted graph using Kruskal's algorithm with Disjoint Set Union data structure.

Pseudocode

ALGO Kruskal(n)

```
SortEdges by weight ascending
InitializeDSU for all vertices
minCost ← 0
FOR each edge (u, v, weight) in sorted order
    IF FindParent(u) ≠ FindParent(v)
        minCost ← minCost + weight
        UnionSet(u, v)
    END IF
END FOR
RETURN minCost
```

Variables Used

- `edges[]` — All graph edges sorted by weight
- `parent[]` — DSU parent array
- `rankValue[]` — DSU rank for union by rank
- `minCost` — Total MST cost

Algorithm Explanation

All edges sorted by weight in ascending order.

For each edge, check if the two endpoints are in different components (using DSU Find).

If different — add edge to MST and union the two components.

If same — skip (would create a cycle). Continues until n-1 edges are in MST.

Use Cases

- Used in **network design** to lay minimum-cost cables connecting all offices or cities in a telecom network.

- Applied in **clustering algorithms** (e.g. single-linkage clustering) to build minimum spanning trees from data points.

- Used in **circuit board design** to connect components with minimum total wire length.

Time Complexity

O(E log E)

Space Complexity

O(V + E)

Program 3: Prim's Algorithm (`Primsalgo.cpp`)

Aim

To find the Minimum Spanning Tree (MST) of a weighted undirected graph using Prim's algorithm with a min-heap starting from vertex 0.

Pseudocode

```
ALGO Prims(n)

    visited[all] ← false

    Push (weight=0, vertex=0) into MinHeap

    minCost ← 0

    WHILE heap not empty

        (weight, u) ← Extract minimum
```

```
        IF visited[u] = true THEN CONTINUE
        visited[u] ← true
        minCost ← minCost + weight
        FOR each neighbor v of u with edge weight w
            IF visited[v] = false
                Push (w, v) into heap
            END IF
        END FOR
    END WHILE
    RETURN minCost
```

Variables Used

- `visited[]` — Tracks vertices already added to MST
- `minCost` — Total MST weight accumulated
- `weight, u` — Edge weight and vertex from heap
- `v, w` — Neighbor and its edge weight

Algorithm Explanation

Start from vertex 0 and greedily grow MST by always picking cheapest edge to an unvisited vertex.

Min-heap ensures minimum weight edge is always processed first.

Mark vertex as visited when included in MST to avoid revisiting.

Continues until all vertices are in the MST.

Use Cases

- Used in **electrical grid planning** to connect all substations with minimum total transmission line length.

- Applied in **road or pipeline network construction** to minimise total infrastructure cost across all connected nodes.

- Used in **image segmentation** where pixels are nodes and edge weights represent intensity differences.

Time Complexity

O(V + E log V)

Space Complexity

O(V + E)

- **WEEK 6:**

Program 1: Cycle Detection in Graph (`cycledetectgraph.cpp`)

Aim

To detect whether a directed graph contains a cycle using Depth First Search with three-state vertex colouring — unvisited, in-stack, and done.

Pseudocode

```
ALGO DFS(node)

   visited[node] ← 1          // IN-STACK

   FOR each neighbor v of node

      IF visited[v] = 1      // back edge = cycle!
```

```
            hasCycle ← true
            RETURN
        END IF
        IF visited[v] = 0      // unvisited
            DFS(v)
        END IF
    END FOR
    visited[node] ← 2          // DONE
ALGO CycleDetection(n)
    visited[all] ← 0
    hasCycle ← false
    FOR i = 0 TO n-1
        IF visited[i] = 0
            DFS(i)
        END IF
    END FOR
    RETURN hasCycle
```

Variables Used

- `adj[][]` — Adjacency matrix of the directed graph

- `visited[]` — State: 0 = unvisited, 1 = in DFS stack, 2 = fully done

- `hasCycle` — Boolean flag, true if cycle is found

- `n` — Number of vertices

Algorithm Explanation

DFS is run from every unvisited vertex.

Each vertex is marked as in-stack (1) when entered and done (2) when all neighbors are processed.

If a neighbor is found already in-stack (state 1), a back edge exists meaning a cycle is detected.

If DFS completes with no back edges found, graph is a valid DAG.

Use Cases

- Used in **operating systems** to detect deadlocks in resource allocation graphs between processes.

- Applied in **build systems** (e.g. Makefile, Gradle) to detect circular dependencies between tasks or modules.

- Used in **compilers** to validate that import/include graphs have no circular references between files.

Time Complexity

O(V + E)

Space Complexity

O(V)

- **<u>WEEK 7:</u>**

<u>Program 1: Multistage Graph - Minimum Cost Path,Stages Count(`Multistagegraph.cpp`)</u>

Aim

To find the minimum cost path from source (vertex 0) to destination (vertex n-1) in a multistage graph using Dynamic Programming forward approach, with automatic stage detection and path display.

PseudocodeALGO AssignStages(n, cost[][])

```
    dist[0] ← 0,  dist[others] ← -1

    FOR u = 0 TO n-1

        IF dist[u] = -1 THEN CONTINUE

        FOR v = 0 TO n-1

            IF edge(u,v) exists AND dist[v] < dist[u]+1

                dist[v] ← dist[u] + 1

        END FOR

    END FOR

    numStages ← dist[n-1] + 1
```

ALGO SolveDP(cost_dp[], next[])

```
    cost_dp[n-1] ← 0        // sink = 0 (base case)

    FOR s = numStages-2 DOWN TO 0

        FOR each vertex u in stage s

            FOR each neighbor v of u

                total ← cost[u][v] + cost_dp[v]

                IF total < cost_dp[u]

                    cost_dp[u] ← total
```

```
                next[u] ← v
            END IF
        END FOR
    END FOR
  END FOR
```

Variables Used

- `n` — Number of vertices
- `e` — Number of edges
- `edges[]` — Edge list storing from, to, cost
- `cost[][]` — Cost adjacency matrix
- `stage[]` — Stage number assigned to each vertex
- `numStages` — Total number of stages detected
- `cost_dp[]` — DP array: min cost from each vertex to sink
- `next[]` — Stores next vertex on optimal path

Algorithm Explanation

Stages are auto-detected using BFS-style forward distance from source vertex 0.

DP is computed backwards from sink (cost_dp[sink]=0) to source.

At each vertex u, minimum cost = min over all neighbors v of { edge(u,v) + cost_dp[v] }.

Path is traced forward from source using next[] array until sink is reached.

Use Cases

- Used in **project scheduling** to find the minimum-cost sequence of stages to complete a multi-phase project.

- Applied in **supply chain management** to compute the cheapest path from production stage to final delivery.

- Used in **network routing** for multi-hop systems where packets must pass through defined intermediate stages.

Time Complexity

O(V + E)

Space Complexity

$O(V^2)$

- **WEEK 8:**

Program 1: Matrix Chain Multiplication (`Matrixchainmultiplication.cpp`)

Aim

To find the optimal parenthesization of a chain of matrices that minimizes the total number of scalar multiplications using Dynamic Programming.

Pseudocode

```
ALGO MatrixChain(p[], n)

   FOR i = 1 TO n

      m[i][i] ← 0        // single matrix = 0 cost

   END FOR

   FOR len = 2 TO n        // increasing chain length
```

```
        FOR i = 1 TO n-len+1
            j ← i + len - 1
            m[i][j] ← ∞
            FOR k = i TO j-1
                cost ← m[i][k] + m[k+1][j] + p[i-1]×p[k]×p[j]
                IF cost < m[i][j]
                    m[i][j] ← cost
                    s[i][j] ← k
                END IF
            END FOR
        END FOR
    END FOR
ALGO PrintParenthesis(i, j)
    IF i = j
        PRINT "M"i
    ELSE
        PRINT "("
        PrintParenthesis(i, s[i][j])
        PrintParenthesis(s[i][j]+1, j)
        PRINT ")"
```

END IF

Variables Used

- `p[]` — Dimension array of size n+1

- `n` — Number of matrices

- `m[][]` — DP cost table storing minimum multiplications

- `s[][]` — Split point table for optimal parenthesization

- `len` — Current chain length being solved

- `k` — Split point being tried

Algorithm Explanation

Fill DP table diagonal by diagonal for increasing chain lengths.

m[i][j] = minimum scalar multiplications to compute product of matrices i through j.

Try every split point k — left part (i..k) and right part (k+1..j) are multiplied.

s[i][j] stores the optimal k for path reconstruction via PrintParenthesis.

Use Cases

- Used in **compiler optimisation** to determine the most efficient order of matrix operations in linear algebra libraries.

- Applied in **machine learning frameworks** (e.g. TensorFlow, PyTorch) to optimise computation graphs for matrix operations.

- Used in **computer graphics pipelines** to minimise cost when chaining multiple transformation matrices.

Time Complexity

$O(n^3)$

Space Complexity

$O(n^2)$

Program 2: Multistage Graph - Forward and Backward Propagation (`Matrixchainmultiplication.cpp`)

Aim

To find the minimum cost path in a multistage graph using both Forward Propagation (source to sink) and Backward Propagation (sink to source) approaches and verify both give the same result.

Pseudocode

```
ALGO ForwardPropagation(n, adj[][])

   dist[0] ← 0,  dist[others] ← 99999

   FOR i = 0 TO n-1

      FOR j = 0 TO n-1

         IF edge(i,j) exists AND dist[i]+adj[i][j] < dist[j]

            dist[j] ← dist[i] + adj[i][j]

            parent[j] ← i

         END IF

      END FOR

   END FOR

   PRINT dist[n-1]
```

```
ALGO BackwardPropagation(n, adj[][])

   dist[n-1] ← 0,  dist[others] ← 99999

   FOR i = n-1 DOWN TO 0

      FOR j = 0 TO n-1

         IF edge(i,j) exists AND dist[j]+adj[i][j] < dist[i]

            dist[i] ← dist[j] + adj[i][j]

            parent[i] ← j

         END IF

      END FOR

   END FOR

   PRINT dist[0]
```

Variables Used

- `adj[][]` — Weighted adjacency matrix

- `dist[]` — Distance/cost array from source or sink

- `parent[]` — Parent array for path reconstruction

- `n` — Number of vertices

- `m` — Number of edges

Algorithm Explanation

Forward: Start dist[source]=0, propagate costs forward — each vertex relaxes its outgoing edges.

Backward: Start dist[sink]=0, propagate costs backward — each vertex relaxes incoming edges.

Both approaches yield the same minimum cost.

Path reconstructed using parent[] array traced to start or end.

Use Cases

- Used in **neural network training** — forward propagation computes output; backward propagation computes gradients.

- Applied in **dynamic programming verification** to cross-check shortest path results from both directions.

- Used in **pipeline scheduling** where tasks must be validated from both start-to-end and end-to-start perspectives.

Time Complexity

$O(V^2)$

Space Complexity

$O(V)$

- **WEEK 9:**

Program 1: Longest Common Subsequence (`lcs.cpp`)

Aim

To find the length and actual string of the Longest Common Subsequence of two given strings using Horowitz and Sahani's Dynamic Programming approach.

Pseudocode

```
ALGO LCSLength(x, y, m, n)

    c[i][0] ← 0  for all i
```

```
    c[0][j] ← 0  for all j
    FOR i = 1 TO m
        FOR j = 1 TO n
            IF x[i] = y[j]
                c[i][j] ← c[i-1][j-1] + 1
                b[i][j] ← 'D'       // Diagonal match
            ELSE IF c[i-1][j] ≥ c[i][j-1]
                c[i][j] ← c[i-1][j]
                b[i][j] ← 'U'       // Up
            ELSE
                c[i][j] ← c[i][j-1]
                b[i][j] ← 'L'       // Left
            END IF
        END FOR
    END FOR
ALGO PrintLCS(x, i, j)
    IF i = 0 OR j = 0 THEN RETURN
    IF b[i][j] = 'D'
        PrintLCS(x, i-1, j-1)
        PRINT x[i]
```

```
    ELSE IF b[i][j] = 'U'
        PrintLCS(x, i-1, j)
    ELSE
        PrintLCS(x, i, j-1)
    END IF
```

Variables Used

- `x, y` — Two input strings

- `m, n` — Lengths of strings x and y

- `c[][]` — DP table storing LCS lengths

- `b[][]` — Direction table: D=diagonal, U=up, L=left

Algorithm Explanation

Fill DP table row by row using the LCS recurrence relation.

If characters match: c[i][j] = c[i-1][j-1] + 1 and mark diagonal.

If no match: take max of cell above or left, mark direction accordingly.

PrintLCS traces back through b[][] from b[m][n] and prints character only on diagonal moves.

Use Cases

- Used in **diff tools** (e.g. `git diff`, Linux `diff`) to highlight differences between two versions of a file.

- Applied in **bioinformatics** to compare DNA or protein sequences and find common genetic patterns.

- Used in **plagiarism detection systems** to find the longest matching sequence between two submitted documents.

Time Complexity

O(m × n)

Space Complexity

O(m × n)

Program 2: Floyd-Warshall All Pairs Shortest Path (`lcs.cpp`)

Aim

To find shortest paths between all pairs of vertices in a weighted directed graph using Floyd-Warshall Dynamic Programming algorithm.

Pseudocode

```
ALGO FloydWarshall(D[][], n)

   FOR i = 0 TO n-1

      D[i][i] ← 0      // self distance = 0

   END FOR



   FOR k = 0 TO n-1         // intermediate vertex

      FOR i = 0 TO n-1     // source vertex

         FOR j = 0 TO n-1 // destination vertex

            IF D[i][k] + D[k][j] < D[i][j]

               D[i][j] ← D[i][k] + D[k][j]

            END IF
```

```
        END FOR
      END FOR
    END FOR
```

Variables Used

- `D[][]` — Distance matrix initialized with input adjacency values

- `n` — Number of vertices

- `k` — Intermediate vertex being considered

- `i` — Source vertex

- `j` — Destination vertex

Algorithm Explanation

Initialize D with direct edge costs (9999 = no edge, 0 = diagonal).

For each intermediate vertex k, try to improve path i→j by going through k.

If D[i][k] + D[k][j] < D[i][j], update D[i][j] with shorter path through k.

After processing all k values, D[i][j] holds shortest path between every pair.

Use Cases

- Used in **network analysis tools** to compute shortest paths between all pairs of routers or servers in a network.

- Applied in **urban traffic planning** to find optimal routes between every pair of intersections in a city map.

- Used in **game map generation** to precompute all-pairs distances for AI navigation in strategy games.

Time Complexity

$O(V^3)$

Space Complexity

$O(V^2)$

- **WEEK 10:**

Program 1: Graph Colouring (`graphcolour.cpp`)

Aim

To assign colours to vertices of a graph such that no two adjacent vertices share the same colour using at most m colours, via backtracking, and measure execution time for different graph sizes.

Pseudocode

```
ALGO ColorGraph(node, V, adj[][], colors[], numColors)

   IF node = V

      RETURN true          // all vertices successfully coloured

   END IF

   FOR c = 1 TO numColors

      IF CanColor(node, c, adj, V, colors)

         colors[node] ← c

         IF ColorGraph(node+1, V, adj, colors, numColors)

            RETURN true

         END IF
```

```
        colors[node] ← 0    // backtrack
      END IF
    END FOR
    RETURN false
ALGO CanColor(node, c, adj[][], V, colors[])
    FOR i = 0 TO V-1
      IF adj[node][i] = 1 AND colors[i] = c
        RETURN false        // conflict with adjacent vertex
      END IF
    END FOR
    RETURN true
```

Variables Used

- `adj[][]` — Adjacency matrix of the graph
- `colors[]` — Colour assigned to each vertex (0 = uncoloured)
- `numColors` — Maximum number of colours allowed (m)
- `node` — Current vertex being coloured
- `c` — Colour currently being tried

Algorithm Explanation

Try each colour 1 to m for the current vertex.

Check if colour is valid — no adjacent vertex has the same colour (CanColor).

If valid, assign colour and recurse for next vertex.

If no valid colour found for a vertex, backtrack and try next colour for previous vertex.

Use Cases

- Used in **compiler register allocation** — variables are graph nodes and colours are CPU registers; adjacent variables need different registers.

- Applied in **exam timetabling** — subjects sharing students are adjacent; colours represent time slots to avoid clashes.

- Used in **frequency assignment** in wireless networks to assign non-interfering frequencies to nearby towers.

Time Complexity

$O(m^n)$

Space Complexity

O(V)

Program 2: Hamiltonian Cycle (`hamiltoniancycle.cpp`)

Aim

To determine if a Hamiltonian Cycle exists in a graph (visiting every vertex exactly once and returning to starting vertex) using backtracking, and measure execution time.

Pseudocode

```
ALGO FindHamiltonian(pos, adj[][], path[], V)

   IF pos = V

      RETURN adj[path[pos-1]][path[0]] = 1   // can we return to start?

   END IF
```

```
    FOR v = 1 TO V-1
        IF IsValid(v, pos, adj, path, V)
            path[pos] ← v
            IF FindHamiltonian(pos+1, adj, path, V)
                RETURN true
            END IF
            path[pos] ← -1      // backtrack
        END IF
    END FOR
    RETURN false
ALGO IsValid(v, pos, adj[][], path[], V)
    IF adj[path[pos-1]][v] = 0  THEN RETURN false   // no edge
    FOR i = 0 TO pos-1
        IF path[i] = v  THEN RETURN false           // already visited
    END FOR
    RETURN true
```

Variables Used

- `adj[][]` — Adjacency matrix of the graph
- `path[]` — Current path being built (stores vertex sequence)
- `pos` — Current position in path being filled

- `V` — Total number of vertices

- `v` — Vertex being tried at current position

Algorithm Explanation

Start from vertex 0 fixed at path[0] and try to fill remaining V-1 positions.

At each position, try all vertices not already in path that have edge from previous vertex.

If all V vertices are placed, check if last vertex connects back to vertex 0 to complete cycle.

Backtrack when no valid vertex found at current position.

Use Cases

- Used in **route planning** for delivery services where every location must be visited exactly once before returning to base.

- Applied in **printed circuit board drilling** to find a path that visits every hole exactly once with minimum backtracking.

- Studied in **operations research** as the core structure of the Travelling Salesman Problem in logistics optimisation.

Time Complexity

O(n!)

Space Complexity

O(n)

Program 3: N-Queens Problem (`nqueen.cpp`)

Aim

To place N queens on an N×N chessboard such that no two queens attack each other (no same row, column, or diagonal), finding and printing all valid solutions using backtracking.

Pseudocode

```
ALGO NQueen(k, n)

   FOR i = 1 TO n

      IF Place(k, i)

         x[k] ← i

         IF k = n

            PRINT solution x[1..n]

         ELSE

            NQueen(k+1, n)      // recurse for next row

         END IF

      END IF

   END FOR

ALGO Place(k, i)

   FOR j = 1 TO k-1

      IF x[j] = i                    // same column conflict

         RETURN false

      END IF

      IF |x[j] - i| = |j - k|         // diagonal conflict

         RETURN false

      END IF
```

```
    END FOR

    RETURN true
```

Variables Used

- `x[]` — x[i] = column position of queen placed in row i

- `k` — Current row being processed

- `n` — Board size (N queens on N×N board)

- `i` — Column position being tried

- `j` — Previous row index for conflict checking

Algorithm Explanation

Place queens one row at a time from row 1 to n.

For each row k, try every column i from 1 to n.

Place() checks: no queen in same column, no queen on same diagonal.

If valid, place queen and recurse for next row. Backtrack if no valid column found.

Prints all valid complete configurations.

Use Cases

- Used in **constraint satisfaction research** as a benchmark problem for evaluating backtracking and heuristic algorithms.

- Applied in **parallel computing** to distribute non-conflicting tasks (queens) across independent processors (rows).

- Used in **AI and game theory** to study how backtracking search explores large solution spaces efficiently.

Time Complexity

O(n!)

Space Complexity

O(n)

Program 4: Sum of Subsets (`sumsubset.cpp`)

Aim

To find all subsets of a given integer array that sum exactly to a given target value using backtracking with pruning.

Pseudocode

```
ALGO FindSubsets(idx, currSum, count, nums[], sz, target, path[])

    IF currSum = target

        PRINT path[0..count-1]     // solution found

        RETURN

    END IF

    IF idx = sz OR currSum > target

        RETURN                     // prune branch

    END IF

    // Include current element

    path[count] ← nums[idx]

    FindSubsets(idx+1, currSum+nums[idx], count+1, nums, sz, target, path)
```

```
    // Exclude current element

    FindSubsets(idx+1, currSum, count, nums, sz, target, path)
```

Variables Used

- `nums[]` — Input array of integers

- `sz` — Total size of the array

- `target` — Required sum to match

- `idx` — Current index being considered

- `currSum` — Running sum of current subset

- `path[]` — Elements selected in current subset

- `count` — Number of elements in current path

Algorithm Explanation

At each element, make two recursive choices: include it or exclude it.

If current sum matches target — a valid subset is found and printed.

If index exceeds array bounds or sum exceeds target — prune this branch (backtrack).

This explores at most 2^n subsets but pruning reduces average case significantly.

Use Cases

- Used in **financial applications** to check if a combination of transactions can exactly match a target amount (e.g. fraud detection).

- Applied in **resource scheduling** to determine if a subset of jobs can exactly fill a given time slot or budget.

- Used in **cryptography and coding theory** to solve subset-sum instances which form the basis of certain knapsack-based encryption schemes.

Time Complexity

$O(2^n)$

Space Complexity

$O(n)$

End of DAA Lab Work README

Reference: Horowitz & Sahani — Fundamentals of Computer Algorithms